

# Lerneinheit 1 · Auftakt: Von Python zur GPU

Version 1.2

## Lernziele

Nach dieser Lerneinheit können Sie ...

- intuitiv erklären, wie eine semantische Suche funktioniert: Embeddings, Kosinus-Ähnlichkeit und die drei Schritte Normalize, Compare, Select,
- die sieben Ebenen, auf denen dieselbe Rechnung laufen kann, grob einordnen und haben ein Gefühl dafür, wie stark sich Laufzeiten unterscheiden können,
- einordnen, woher Python kommt, wie es ausgeführt wird und warum es die Sprache der KI-Welt ist,
- einfache Python-Funktionen lesen, schreiben, ihr Verhalten erklären und fehlgeschlagene Prüfzellen zur Korrektur nutzen.

Diese Lerneinheit besteht aus zwei Teilen. Der erste ordnet ein: Worum geht es in diesem Modul, und wie ist es organisiert? Ein lauffähiger Appetizer gibt den Vorgeschmack, und ein kurzes Kapitel stellt die Sprache Python und ihr Ökosystem vor. Der zweite Teil legt los: das Lab, in dem Sie Ihre ersten eigenen Python-Funktionen auf echten Daten schreiben. Die Sprache selbst führen die Python-Grundlagen (`LE01_python_grundlagen_1.ipynb`) ein, kompakt und für alle, die schon programmieren können (in C++ aus PROG1).

## Inhalt

- 1.1 Worum es in diesem Modul geht
  - 1.1.1 Der Appetizer: Semantische Suche
  - 1.1.2 Wie kommt Bedeutung in Zahlen?
  - 1.1.3 Eine Rechnung, sieben Ebenen
  - 1.1.4 Messen ist schwerer, als es aussieht
  - 1.1.5 Die Landkarte: von Python zur GPU
- 1.2 Organisation und Arbeitsweise
- 1.3 Python und sein Ökosystem M1
- 1.4 Lab: Funktionen für die Suche M1
  - 1.4.1 Die Daten
  - 1.4.2 Treffer formatieren
  - 1.4.3 Treffer filtern
  - 1.4.4 Die besten Treffer finden
  - 1.4.5 Alles zusammensetzen
  - 1.4.6 Zusatzaufgaben (Fast Track)

## 1.1 Worum es in diesem Modul geht

Jeder kann heute in wenigen Zeilen ein neuronales Netz trainieren, denn die Werkzeuge sind so gut geworden, dass man nicht mehr verstehen muss, was dabei passiert. Genau dieses Verstehen ist das Ziel dieses Moduls. Anhand eines kleinen Beispiels, einer **semantischen Suche**, verfolgen wir den KI-Stack durch alle Schichten. Ihre drei Rechenschritte (Normalize, Compare, Select) bauen Sie im Lauf des Semesters erst in NumPy, dann in PyTorch und am Ende als eigenen GPU-Kernel, also als kleines Programm, das direkt auf der GPU läuft. Bei jedem Schritt vergleichen Sie die Laufzeiten und verstehen, woher die Unterschiede kommen. Wie groß sie ausfallen, hängt von Datenmenge, Hardware und Implementierung ab, und genau dieses Beurteilen lernen Sie hier.

### Um Missverständnisse zu vermeiden: Mit KI oder für KI programmieren?

**Programmieren mit KI** heißt: mit KI-Assistenten Code schreiben. Das dürfen Sie hier gerne in den Übungen nutzen, nur nicht in der Klausur.

**Programmieren für KI** heißt: den Code verstehen und schreiben, der KI-Systeme trägt — Tensoren, Operatoren, Kernel. Darum geht es in diesem Modul; was diese Begriffe genau bedeuten, klärt das Semester Stück für Stück.

**Warum dieses Modul?** Fünf Gründe:

- **Python ist das Werkzeug, das Sie überall brauchen.** Datenanalyse, Automatisierung, Prototypen, Abschlussarbeit — kaum ein Projekt kommt heute ohne Python, NumPy und pandas aus. Diesen Werkzeugkasten nehmen Sie auf jeden Fall mit, auch wenn Sie später nie mehr einen GPU-Kernel anfassen.
- **KI benutzen kann jeder — verstehen die wenigsten.** Drei Zeilen PyTorch trainieren ein Netz. Was zwischen diesen Zeilen und der Hardware passiert, wissen nur wenige — genau das lernen Sie hier.
- **Performance ist wieder ein Thema.** KI-Rechenzeit kostet echtes Geld. Wer beurteilen kann, warum etwas langsam ist und was es schneller macht, ist in jedem KI-Projekt wertvoll.
- **Sie sehen den ganzen Stack einmal von oben bis unten.** Anhand eines Beispiels begleitet Sie das Modul von der Python-Zeile bis zum eigenen GPU-Kernel — am Ende gibt es keine Blackbox mehr dazwischen.
- **Eine gute Basis für das, was danach kommt.** Abschlussarbeiten mit KI-Anteil, Berufseinstieg im Data- und KI-Umfeld — überall zählt genau dieses Fundament ein.

„**Macht das heutzutage nicht einfach die KI?**“ Ein berechtigter Einwand, und ein altes Muster. Seit es Compiler gibt, schreibt kaum noch jemand Assembler: Der Code in einer Hochsprache wird automatisch in Maschinensprache übersetzt. LLMs setzen jetzt eine weitere Abstraktionsebene obendrauf und übersetzen Anforderungen und Arbeitsanweisungen in natürlicher Sprache in Code. Aber als Informatikerin oder Informatiker müssen Sie für Ihren Code die Hand ins Feuer legen können, statt blind einem Coding-Agenten zu vertrauen. Genau deshalb geht dieses Modul bewusst in die andere Richtung: nach unten, in die Ebenen unter Ihrem Code. Dort verstehen Sie, was passiert, und programmieren es selbst.

Wie sich das anfühlt, zeigt gleich der Appetizer. Verstehen müssen Sie darin noch nichts.

#### 1.1.1 Der Appetizer: Semantische Suche

Eine Suchmaschine, die Bedeutung versteht, und der Blick auf die sieben Ebenen, auf denen sie rechnet. Sie geben eine Frage ein und bekommen die passenden Wikipedia-Absätze, obwohl kein einziges Wort der Frage darin vorkommen muss. Wie das funktioniert, ist ein durchgängiges Beispiel in diesem Modul. Heute schauen Sie nur zu, drehen an den Eingaben und sehen, wie unterschiedlich schnell dieselbe Rechnung auf sieben Ebenen läuft. Die Python-Konstrukte in den Zellen müssen Sie noch nicht selbst schreiben können — die Kreisziern erklären sie, und geübt wird im Lab.

Unsere Suchmaschine besteht aus zwei Zutaten, die im Ordner `data/` liegen und die die nächste Zelle bei Bedarf automatisch nachlädt:

- **400 Absätze aus der deutschen Wikipedia:** unser Dokumentbestand, der „Korpus“. Jeder Absatz stammt aus einem anderen Artikel.
- **Je Absatz ein Vektor aus 384 Zahlen:** die sogenannten Embeddings. Ein Sprachmodell hat jeden Absatz vorab in einen solchen Zahlenvektor übersetzt; die Idee dahinter zeigt der Abschnitt [1.1.2](#).

Dazu kommen acht **Beispielfragen** (passende Fragen zu diesem Korpus), ebenfalls schon als Text und als Vektor. So funktioniert die Suche auch ohne installiertes Sprachmodell.

```

① import json
   import numpy as np
② # Korpus laden: die Absätze als Text und die zugehörigen Vektoren als Matrix.
   # E hat die Form [400, 384]: eine Zeile je Absatz, 384 Zahlen je Zeile.
   corpus = json.load(open("data/appetizer_passages.json"))
   passages = corpus["passages"]
③ E = np.load("data/appetizer_embeddings.npy")

   # Die Beispielanfragen: ebenfalls Text plus je ein Vektor (Matrix Q, Form [8, 384]).
   queries = json.load(open("data/appetizer_queries.json"))["queries"]
   Q = np.load("data/appetizer_query_embeddings.npy")
④ print(len(passages), "Passagen ·", E.shape[1], "Zahlen je Vektor ·", len(queries), "Beispielanfragen")

```

- ① `import` lädt ein Modul, also eine fertige Werkzeugsammlung; `as np` gibt ihm einen kurzen Spitznamen, unter dem wir es ab jetzt ansprechen
- ② `json.load` liest eine JSON-Datei in ein Dictionary, eine Zuordnung von Namen zu Werten; `["passages"]` greift den Eintrag mit diesem Namen heraus
- ③ `np.load` liest ein gespeichertes NumPy-Array von der Platte — hier die Matrix mit den 400 Vektoren
- ④ `print` nimmt beliebig viele Werte, durch Kommas getrennt, und gibt sie mit Leerzeichen verbunden aus
- 400 Passagen · 384 Zahlen je Vektor · 8 Beispielanfragen

Werfen wir zuerst einen Blick in die Daten. So sehen drei der 400 Absätze aus. Jeder hat eine Nummer (`id`), einen Artikeltitel und den eigentlichen Text:

```

# Drei Beispiel-Absätze anzeigen (von 400)
① for p in passages[:3]:
②     print(f"[{p['id']}] {p['title']}")
③     print("    " + p["text"][:150].replace("\n", " ") + " ...\n")

```

- ① die `for`-Schleife läuft direkt über die Elemente der Liste, ohne Index; `[:3]` schneidet vorher die ersten drei heraus (ein „Slice“)
- ② f-String: das `f` vor dem String erlaubt, Ausdrücke in `{}` direkt in den Text einzusetzen
- ③ Methoden lassen sich verketteten und wirken von links nach rechts: erst kürzt `[:150]` den Text, dann ersetzt `.replace` die Zeilenumbrüche durch Leerzeichen

```

[0] Aufzugsanlage
    Seilloser Aufzug. An der RWTH Aachen im Institut für Elektrische Maschinen wurde ein seilloser
    Aufzug entwickelt und ein Prototyp aufgebaut. Die Kabin ...

[1] Sichuan
    Landwirtschaft. Sichuan gilt vorwiegend als Reisanbaugebiet, ist aber auch bedeutender Produzent
    von Mais, Süßkartoffeln, Weizen, Raps und Soja. Der A ...

[2] Antenne
    Polarisation. Antennen strahlen polarisierte Wellen ab. Als Polarisationsebene wurde die Richtung
    des elektrischen Feldvektors gewählt. So schwingt be ...

```

Und das sind die acht Beispielanfragen. Die Nummer vorne brauchen Sie gleich, um eine Anfrage auszuwählen:

```

# Alle Beispielanfragen mit ihrer Nummer
① for no, q in enumerate(queries):
    print(f"[{no}] {q['text']}")

```

- ① `enumerate` liefert beim Durchlaufen Nummer und Element zugleich
- ```

[0] Wo liegt San Diego?
[1] Wer hat das Logo von Windows 8 entworfen?
[2] Wie werden die NPOs finanziert?
[3] Wie viel wiegt das Gehirn eines Mannes durchschnittlich?
[4] Auf was beruhen Browserdatenstatistiken?
[5] Durch was kann Zeit in der Musik erfahrbar gemacht werden?
[6] Was ist armenische Kunst?
[7] Was sind die wichtigsten Abschnitt Maßeinheits Größen des Korans?

```

Texte mit ähnlicher Bedeutung bekommen vom Sprachmodell ähnliche Vektoren: „Wo liegt San Diego?“ und ein Absatz über die Geographie San Diegos zeigen im 384-dimensionalen Raum in fast dieselbe Richtung. Suchen heißt deshalb nur noch, den Vektor der Frage mit allen 400 Absatz-Vektoren zu vergleichen und die ähnlichsten herauszusuchen. Drei Schritte: **Normalize** (alle Vektoren auf Länge 1 bringen, damit nur die Richtung zählt, nicht die Länge), **Compare** (Skalarprodukt des Frage-Vektors mit jedem Absatz-Vektor; bei Länge 1 ist das genau die Kosinus-Ähnlichkeit: 1 = gleiche Richtung, 0 = senkrecht zueinander), **Select** (die k besten Treffer nehmen).

Warum das Normalisieren dazugehört: Ohne diesen Schritt würde die Länge der Vektoren das Ergebnis verzerren, und ein Absatz könnte allein durch betragsmäßig große Zahlen „ähnlicher“ wirken als sein Inhalt rechtfertigt. Nach dem Normalisieren ist das Skalarprodukt genau der Kosinus des Winkels zwischen den beiden Richtungen, und nur die Richtung kodiert die Bedeutung. Das unterscheidet diese Suche auch von der klassischen Stichwortsuche: Verglichen werden nicht Buchstabenfolgen, sondern Bedeutungen. „Wo liegt San Diego?“ findet deshalb auch Absätze, die keines der Frageworte enthalten, solange sie inhaltlich passen. Genau diese drei Schritte begleiten Sie als Semesterfall durch alle Lerneinheiten: erst in NumPy, später in PyTorch, am Ende als eigener GPU-Kernel. Ein erstes intuitives Bild, wie ein Sprachmodell zu seinen Vektoren kommt und was die Kosinus-Ähnlichkeit misst, gibt gleich der Abschnitt [1.1.2](#).

Die folgende Zelle setzt diese drei Schritte in wenigen Zeilen um. Die Funktion `search(query_no, k)` hat zwei Parameter: `query_no` ist die **Nummer der Beispielfrage** aus der Liste oben (0 bis 7), und `k` bestimmt, **wie viele Treffer** angezeigt werden. Hinter jedem Treffer steht sein Ähnlichkeitswert. Je näher an 1, desto besser passt der Absatz zur Frage:

```

① def normalize(v):
    # Jede Zeile durch ihre Länge teilen -> alle Vektoren haben Länge 1
    return v / np.linalg.norm(v, axis=-1, keepdims=True)
② def search(query_no, k=3):
③     q = normalize(Q[query_no])           # Normalize: der Fragevektor
    scores = normalize(E) @ q              # Compare: Kosinus-Ähnlichkeit zu allen 400 Absätzen
④     best = np.argsort(scores)[::-1][:k]   # Select: die k höchsten Werte
⑤     print("Anfrage:", queries[query_no]["text"], "\n")
    for pid in best:
        p = passages[pid]
        print(f" {scores[pid]:.3f} {p['title']}")
        print("      " + p["text"][:160].replace("\n", " ") + " ...\n")
⑥ # Anfrage Nr. 0 ("Wo liegt San Diego?"), drei Treffer:
    search(0)

```

- ① `def` definiert eine Funktion: keine Typangaben, der Rumpf ist eingerückt statt geklammert, `return` gibt das Ergebnis zurück
- ② `k=3` gibt dem Parameter einen Standardwert: Wer beim Aufruf nichts angibt, bekommt automatisch drei Treffer
- ③ Variablen entstehen durch bloße Zuweisung, ohne Deklaration; `@` ist der Operator für Matrixmultiplikation — hier rechnet er alle 400 Skalarprodukte auf einen Schlag
- ④ Slicing mit Schritten: `[::-1]` dreht die sortierte Reihenfolge um, `[:k]` nimmt davon die ersten k — zusammen die k höchsten Werte (mehr dazu in der nächsten Lerneinheit)
- ⑤ `:.3f` hinter dem Doppelpunkt im f-String formatiert den Wert mit genau drei Nachkommastellen
- ⑥ der Aufruf: `k` bleibt weg, also gilt der Standardwert 3 von oben

Anfrage: Wo liegt San Diego?

0.831 San Diego

Geographie. San Diego liegt an einem künstlichen Hafenbecken an der Südspitze Kaliforniens, etwa zweieinhalb Autostunden südlich von Los Angeles und etwa eine h ...

0.351 Mexiko-Stadt

Klima. Die Stadt befindet sich in den Tropen, genauer gesagt in den Kalttropen wegen ihrer hohen Lage. Tagsüber ist es im Winter mit 20 bis 25 °C recht warm, na ...

0.338 North Carolina

Regionen. Geografisch und geologisch lässt sich North Carolina von Ost nach West in drei wesentliche Teile gliedern: die Küstenebene am Atlantik, das Piedmont-P ...

```
# Probieren Sie es selbst: andere Anfrage-Nummer (0 bis 7), mehr Treffer über k.
# Hier: Anfrage Nr. 3 mit den fünf besten Treffern.
```

```
① search(3, k=5)
```

① Argumente lassen sich beim Aufruf benennen: `k=5` setzt gezielt den Parameter `k` und überschreibt dessen Standardwert

Anfrage: Wie viel wiegt das Gehirn eines Mannes durchschnittlich?

0.895 Gehirn

Gehirne von Männern und Frauen. Die Gehirne von Männern und Frauen unterscheiden sich in der Größe und im Aufbau. Durchschnittlich wiegt das Gehirn eines erwach ...

0.456 Papier

Geometrische Eigenschaften. Die Masse (bzw. umgangssprachlich auch das ''Gewicht'') von Papier wird meist flächenbezogen angegeben – konkret in Gramm pro Quadra ...

0.360 Seattle

Demografie. Die Bevölkerung bestand laut dem Zensus von 2010 zu 66,3 Prozent aus Weißen und zu 7,9 Prozent aus Afroamerikanern; 13,8 Prozent waren asiatischer H ...

0.353 Montana

Wirtschaft und Infrastruktur. Das reale Bruttoinlandsprodukt pro Kopf (engl. per capita GDP) – der wichtigste Wohlstandsindikator – lag im Jahre 2016 bei USD 44 ...

0.352 Vögel

Vogelintelligenz. Forschungsergebnisse seit den 1990er Jahren erbrachten, dass Vögel, zumindest aber die Vertreter bestimmter Großgruppen, intelligenter sind, a ...

Achten Sie beim Ausprobieren auf zwei Dinge: Der beste Treffer hat meist einen deutlich höheren Ähnlichkeitswert als der Rest, liegt nach diesem Maß also klar vorn. Das ist eine Aussage über Ähnlichkeit, keine Garantie, dass er die Frage richtig beantwortet — ein Beispiel dafür zeigt der Exkurs unten. Und die Treffer teilen oft kein einziges Wort mit der Frage, denn verglichen wird die Bedeutung (Semantik), nicht einzelne Buchstaben oder Strings.

### Exkurs: Eigene Fragen stellen

Die Beispielanfragen sind vorberechnet, damit dieses Notebook ohne Sprachmodell läuft. Wenn Sie das Modell dazuholen, können Sie frei fragen. Die erste der beiden folgenden Zellen richtet alles ein, die zweite ist Ihre Spielwiese.

Was in der Setup-Zelle passiert: `model.encode` verwandelt Ihre Frage in genau so einen 384-dimensionalen Vektor, wie sie in unserer Matrix `E` liegen. Danach läuft exakt die Pipeline von oben: Normalize, Compare, Select. Neu ist also nur der erste Schritt, der Text zu Zahlen macht — und der braucht das Sprachmodell. Beim ersten Ausführen lädt es rund 500 MB herunter; die vielen Fortschrittsbalken und eine Warnung über „unauthenticated requests“ zum Hugging Face Hub sind dabei normal und kein Fehler. In Colab ist das in etwa einer Minute erledigt; auf dem eigenen Rechner lohnt es nur mit guter Verbindung.

Achten Sie beim Spielen auf eine Eigenheit: Die Suche liefert immer das Ähnlichste im Korpus, nicht die Antwort. „Wer erfand die Buchdruckkunst?“ trifft hier „Papier“ mit 0.698 — einen Gutenberg-Artikel gibt es im Mini-Korpus nicht, also gewinnt der thematisch nächste. Eine semantische Suche kann nur finden, was da ist; ob das Gefundene die Frage beantwortet, muss der Nutzer (oder in einem RAG-System das Sprachmodell) selbst beurteilen.

```
# Setup – in Colab die Kommentarseichen entfernen und EINMAL ausführen:
# %pip install -q sentence-transformers
# from sentence_transformers import SentenceTransformer
# model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
#
# def ask(text, k=3):
#     q = normalize(model.encode([text])[0]) # Ihre Frage -> Embedding, gleicher Raum wie E
#     scores = normalize(E) @ q               # Normalize + Compare, wie oben
#     for pid in np.argsort(scores)[-1][:k]: # Select: die k besten Treffer
#         print(f" {scores[pid]:.3f} {passages[pid]['title']}")

# Spielwiese – Kommentarseichen entfernen, eigene Fragen stellen, k variieren:
# ask("Wer erfand die Buchdruckkunst?")
# ask("Warum ist der Himmel blau?", k=5)
```

### 1.1.2 Wie kommt Bedeutung in Zahlen?

Die Suche eben hat mit „Vektoren aus 384 Zahlen“ gerechnet. Zeit für die Frage, die sich dabei aufdrängt: Wie kommt die **Bedeutung** eines Textes in Zahlen? Ein neuronales Netz rechnet mit Zahlen, nicht mit Buchstaben. Mit dem Satz „San Diego liegt am Pazifik“ kann es zunächst nichts anfangen. Irgendeine Übersetzung von Text in Zahlen muss also her, und wie man sie wählt, entscheidet darüber, was das Netz lernen kann. Ein intuitives Bild davon genügt für heute.

Der naheliegende erste Versuch: ein Vokabular nehmen (sagen wir den Duden) und die Wörter durchnummerieren. „Hund“ bekommt die 2.317, „Katze“ die 3.589. Damit das Netz die Nummern nicht fälschlich als Größen versteht (3.589 ist nicht „mehr“ als 2.317), schreibt man sie meist als **One-Hot-Vektor**: ein Vektor, so lang wie das Vokabular, mit einer 1 an der Position des Wortes und sonst lauter Nullen. Das funktioniert, hat aber zwei Probleme. Die Vektoren sind riesig (eine Position je Wort im Vokabular). Und vor allem: Sie enthalten keinerlei Bedeutung. „Hund“ und „Katze“ sind in dieser Darstellung genauso verschieden wie „Hund“ und „Auto“ — jedes Wort steht allein. Gesucht ist also eine Darstellung, in der ähnliche Bedeutung auch ähnliche Zahlen ergibt.

Woher soll eine Maschine wissen, was ein Wort bedeutet? Die Antwort, auf der moderne Sprachmodelle beruhen, ist verblüffend einfach: **aus dem Kontext**. Schauen Sie, was in diese Lücken passt:

- „Ich trinke morgens eine Tasse \_\_\_\_.“ — *Tee, Kaffee* (aber nicht *Auto*)
- „Der \_\_\_\_ bellt jede Nacht im Garten.“ — *Hund* (vielleicht noch *Wachhund*, sicher keine *Katze*)
- „Wir fahren mit dem \_\_\_\_ in den Urlaub.“ — *Auto, Zug, Wohnmobil*

Wörter, die in denselben Lücken stehen können, also in ähnlichen Kontexten auftauchen, haben ähnliche Bedeutung. Ein Sprachmodell liest Milliarden von Sätzen und lernt daraus für jedes Wort (und zusammengesetzt für ganze Absätze) einen Zahlenvektor, sodass ähnliche Kontexte zu ähnlichen Vektoren führen. Bedeutung steckt dann nicht in einer einzelnen Zahl, sondern in der **Richtung**, in die der Vektor zeigt.

Die Idee ist älter als die heutige KI: „You shall know a word by the company it keeps“ (J. R. Firth, 1957). Eine einzelne Zahl je Wort würde übrigens nicht reichen, denn Bedeutung hat viele Facetten (Lebewesen oder Gegenstand? konkret oder abstrakt? positiv oder negativ?). In echten Embeddings liegt eine solche Facette allerdings nicht in einer einzelnen, benennbaren Dimension, sondern verteilt über viele Koordinaten zugleich — auf „die Obst-Dimension“ kann im Vektor niemand zeigen. Als stark vereinfachtes Denkbild hilft sie trotzdem: Ein mehrdeutiges Wort wie „Apple“ braucht Anteile in Richtung Obst und in Richtung Firma. Moderne Sprachmodelle lösen Mehrdeutigkeit zusätzlich über den Kontext auf: Sie berechnen den Vektor nicht je Wort, sondern je Verwendung — „Apple“ im Obstsalat-Satz bekommt einen anderen Vektor als „Apple“ in der Quartalsbilanz.

**Definition: Embedding**

Ein **Embedding** ist ein gelernter Zahlenvektor, der ein Objekt — ein Wort, einen Satz, einen Absatz — so in einem Raum platziert, dass **ähnliche Bedeutung zu ähnlichen Vektoren** führt. Typisch sind einige hundert Dimensionen; unser Kursmodell verwendet 384.

In diesem Raum lässt sich sogar rechnen: Das berühmteste Beispiel der Embedding-Forschung ist *König – Mann + Frau ≈ Königin*. Zieht man vom König-Vektor den Mann-Vektor ab, bleibt so etwas wie „Herrscher ohne Geschlecht“ übrig; addiert man den Frau-Vektor, landet man in der Nähe von „Königin“. Die Richtung „männlich → weiblich“ ist im Vektorraum also tatsächlich eine Richtung, und sie ist für König/Königin dieselbe wie für Mann/Frau. Wie solche Strukturen beim Lernen entstehen, zeigt anschaulich die Vertiefung am Ende dieses Abschnitts.

Ein Spielzeugbeispiel mit nur vier Dimensionen macht das greifbar. Wir geben drei Wörtern von Hand kleine Vektoren (*Hund* und *Katze* bekommen ähnliche Zahlen, *Auto* ganz andere) und messen die Ähnlichkeit dann genauso, wie es die Suche oben tut: mit der **Kosinus-Ähnlichkeit**.

```
# Spielzeug-Embeddings: vier Dimensionen statt 384 (die Werte sind frei erfunden)
① hund = np.array([0.70, -0.20, 0.50, 0.90])
katze = np.array([0.55, -0.10, 0.65, 0.90])
auto = np.array([0.10, 0.70, -0.20, 0.30])

# Kosinus-Ähnlichkeit: beide Vektoren auf Länge 1 bringen, dann Skalarprodukt
# (setzt Vektoren ungleich dem Nullvektor voraus, sonst Division durch null)
def cos_sim(a, b):
    return (a / np.linalg.norm(a)) @ (b / np.linalg.norm(b))

print(f"cos(Hund, Katze) = {cos_sim(hund, katze):.2f}")
print(f"cos(Hund, Auto) = {cos_sim(hund, auto):.2f}")
```

① `np.array` macht aus einer Python-Liste einen NumPy-Vektor, mit dem sich mathematisch rechnen lässt (elementweise und als Ganzes)

```
cos(Hund, Katze) = 0.98
cos(Hund, Auto) = 0.10
```

**Probieren Sie es selbst:** Ändern Sie oben die Werte von `auto` so, dass die Ähnlichkeit zu *Hund* negativ wird. Überlegen Sie zuerst, welche Vorzeichen Sie dafür kippen müssen, und prüfen Sie Ihre Erwartung dann durch erneutes Ausführen der Zelle.

## Kosinus-Ähnlichkeit

Gegeben seien zwei Vektoren  $A$  und  $B$ , beide ungleich dem Nullvektor (sonst wäre der Nenner null). Ihre Kosinus-Ähnlichkeit ist der Kosinus des Winkels  $\theta$  zwischen ihnen:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

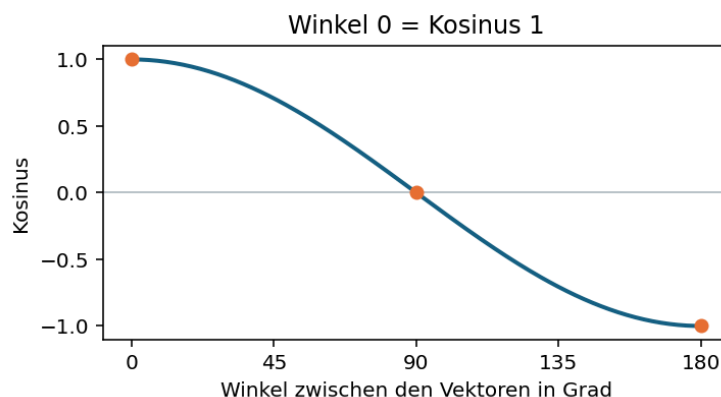
Der Zähler ist das Skalarprodukt, ausgeschrieben  $A \cdot B = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$ : Jede Dimension trägt das Produkt der beiden Werte bei. Haben beide Vektoren in denselben Dimensionen hohe Werte, werden diese Beiträge groß, und die Summe auch. Der Nenner  $\|A\| \|B\|$  teilt durch die Längen der Vektoren (die euklidischen Normen) und macht das Ergebnis damit von der Länge unabhängig. Gemessen wird so der Winkel zweier Richtungen: Der Wert 1 heißt gleiche Richtung, 0 senkrecht zueinander, -1 entgegengesetzt — das sind zunächst rein geometrische Aussagen. In Bedeutung übersetzt: Ein hoher Wert heißt, das Modell hält die Texte für sehr ähnlich; ein Wert nahe 0 oder darunter heißt nur, dass das Modell wenig Verbindung sieht. Sprachliche Gegensätze liegen dabei nicht bei -1: „heiß“ und „kalt“ tauchen in denselben Kontexten auf und bekommen darum sogar recht ähnliche Vektoren. Mathematisch liegt der Wert immer zwischen -1 und 1. Auch negative Werte kommen vor — in unseren Kursdaten bei gut einem Viertel der Frage-Absatz-Vergleiche, wenn auch nie unter -0,23; wie häufig und wie tief, hängt vom Modell und den verglichenen Texten ab.

Bringt man beide Vektoren vorher auf Länge 1, wird der Nenner zu 1, und vom Bruch bleibt nur das Skalarprodukt übrig. Genau deshalb besteht unsere Suche aus **Normalize** und dann **Compare**: Das ist diese Formel, arbeitsteilig aufgeschrieben.

Warum ergibt ein Winkel von 0 genau den Wert 1? Ein Blick auf die Kosinus-Kurve zeigt es: Bei 0° startet sie bei 1, fällt bis 90° auf 0 und erreicht bei 180° den Wert -1:

```
# Die Kosinus-Kurve: von Winkel 0 (gleiche Richtung) bis 180 Grad (entgegengesetzt)
① import matplotlib.pyplot as plt
② winkel = np.linspace(0, np.pi, 200)
③ fig, ax = plt.subplots(figsize=(5.5, 2.6))
  ax.plot(np.degrees(winkel), np.cos(winkel), color="#156082", lw=2)
④ for grad in [0, 90, 180]:
    ax.scatter([grad], [np.cos(np.radians(grad))], color="#E97132", zorder=3)
  ax.axhline(0, color="#adb5bd", lw=0.8)
⑤ ax.set_xticks([0, 45, 90, 135, 180])
  ax.set_xlabel("Winkel zwischen den Vektoren in Grad")
  ax.set_ylabel("Kosinus")
  ax.set_title("Winkel 0 = Kosinus 1")
  plt.show()
```

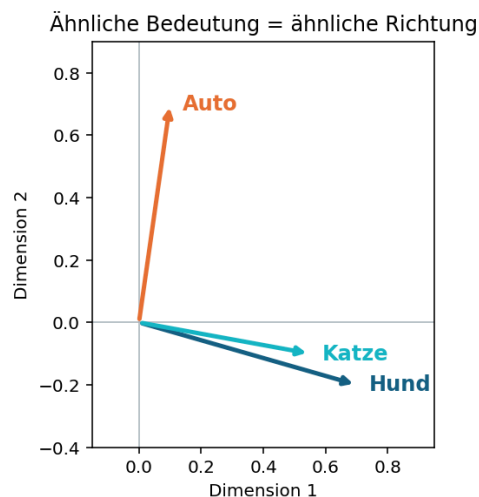
- ① matplotlib ist die Standard-Bibliothek zum Zeichnen; richtig eingeführt wird sie in der pandas/Matplotlib-Lerneinheit
- ② `np.linspace` erzeugt 200 gleichmäßig verteilte Winkel zwischen 0 und pi — `np.cos` rechnet dann für alle 200 auf einmal
- ③ `plt.subplots()` liefert Zeichenfläche (`fig`) und Achsen (`ax`); `ax.plot` zeichnet die Kurve, die x-Achse in Grad
- ④ die drei Merkpunkte 0, 90 und 180 Grad als Punkte; `axhline` zieht die Nulllinie
- ⑤ Achsenbeschriftung und Titel; `plt.show()` zeigt die fertige Figur an



Und weil „Richtung“ das entscheidende Bild ist, zeichnen wir auch unsere Spielzeug-Vektoren. Für die Skizze nehmen wir nur die ersten zwei der vier Dimensionen. Das ist eine Projektion, und die kann Winkel verändern — die Winkel im Bild entsprechen also nicht exakt den eben berechneten vierdimensionalen Werten. Das Prinzip zeigt sie trotzdem: *Hund* und *Katze* zeigen fast in dieselbe Richtung, *Auto* zeigt woandershin:

```
# Die Spielzeug-Embeddings als Pfeile (Dimension 1 und 2)
① fig, ax = plt.subplots(figsize=(4.2, 4.2))
② for name, v, farbe in [("Hund", hund, "#156082"), ("Katze", katze, "#16B8C6"),
                        ("Auto", auto, "#E97132")]:
    ax.annotate("", xy=(v[0], v[1]), xytext=(0, 0),
               arrowprops=dict(arrowstyle="->", lw=2.5, color=farbe))
    ax.text(v[0] + 0.04, v[1], name, color=farbe, fontsize=12, fontweight="bold", va="center")
③ ax.axhline(0, color="#adb5bd", lw=0.8)
   ax.axvline(0, color="#adb5bd", lw=0.8)
   ax.set_xlim(-0.15, 0.95); ax.set_ylim(-0.4, 0.9)
   ax.set_aspect("equal")
   ax.set_xlabel("Dimension 1"); ax.set_ylabel("Dimension 2")
   ax.set_title("Ähnliche Bedeutung = ähnliche Richtung")
   plt.show()
```

- ① quadratische Zeichenfläche; je Vektor ein Eintrag aus Name, Werten und Farbe
- ② `ax.annotate` zeichnet einen Pfeil vom Ursprung (`xytext`) zur Vektorspitze (`xy`), `ax.text` setzt den Namen daneben
- ③ Nulllinien und `set_aspect("equal")`: gleiche Skala auf beiden Achsen, sonst verzerrt das Bild die Winkel



Echte Embeddings mit ihren Hunderten Dimensionen lassen sich so nicht direkt zeichnen. Dafür gibt es Verfahren, die den hochdimensionalen Raum auf zwei Dimensionen zusammendrücken und dabei die Nachbarschaften möglichst erhalten (PCA, t-SNE, UMAP). Solche Karten, auf denen sich verwandte Begriffe zu Inseln gruppieren, sehen Sie im Data-Science-Teil des Moduls.

### Exkurs: RAG — wo Ihnen das wieder begegnet

Wenn ein Chatbot Fragen „zu Ihren eigenen Dokumenten“ beantwortet, zum Firmenwissen etwa oder zu Vorlesungsunterlagen, steckt dahinter fast immer **RAG** (*Retrieval-Augmented Generation*): Bevor das Sprachmodell antwortet, sucht das System die passenden Textstellen heraus und legt sie ihm mit in die Anfrage. Dieses Heraussuchen, der Retrieval-Teil, ist exakt unsere semantische Suche: Embeddings vergleichen per Kosinus-Ähnlichkeit, nur über Millionen statt 400 Absätze. Was Sie in diesem Modul bauen, ist also kein theoretisches Spielzeug, sondern der Rechenkern einer der aktuell wichtigsten KI-Architekturen der Praxis.

 **Vertiefung:** J. Alamar: [The Illustrated Word2vec](#) — anschauliche, bebilderte Einführung, wie Embeddings gelernt werden und warum König – Mann + Frau ≈ Königin funktioniert. Wer tiefer in kontextabhängige Embeddings einsteigen will: [Peters u. a., Deep contextualized word representations, 2018](#) — ein wissenschaftliches Originalpaper und damit deutlich schwerere Kost; fürs Modul genügt die Idee.

### 1.1.3 Eine Rechnung, sieben Ebenen

Bei 400 Absätzen ist die Suche in jedem Fall schnell. Echte Systeme durchsuchen aber Millionen von Dokumenten, und dann wird ein Schritt teuer: das Normalisieren und Vergleichen riesiger Zahlenmengen. Die Mathematik dahinter ist simpel: Jeder Vektor, also jede Zeile der Matrix, wird durch seine Länge geteilt. Aber ein Computer kann genau diese Rechnung auf sehr unterschiedliche Weise ausführen, und die Unterschiede sind gewaltig. Das schauen wir uns jetzt an.

Sieben Ebenen, von der handgeschriebenen Python-Schleife bis zum eigenen GPU-Kernel. Was sie unterscheidet, ist vor allem, **wer die Schleife ausführt**: der Python-Interpreter Zeile für Zeile, eine vorkompilierte Bibliothek wie NumPy in Maschinencode, ein Compiler, der den Code erst zur Laufzeit übersetzt, oder Tausende GPU-Threads gleichzeitig. Die Rechnung selbst bleibt dabei identisch, nur der Weg vom Python-Text zur ausgeführten Arithmetik ändert sich — und zwischen der obersten und der untersten Ebene liegt ein Faktor im Tausenderbereich, den Sie gleich an echten Messwerten sehen. Jede Ebene ist eine eigene Station dieses Semesters. Für heute genügt eine grobe Orientierung: die Ebenen wiedererkennen und die Größenordnungen einordnen können.

Damit der Vergleich fair ist, messen alle Ebenen dieselbe Aufgabe auf denselben Daten: einem **synthetischen Messsatz** aus 10.000 Zufallsvektoren mit je 384 Zahlen, aufgebaut wie unser Korpus, nur 25-mal größer (insgesamt 3,84 Millionen Zahlen). Synthetisch deshalb, weil Zufallsdaten mit festem Seed jederzeit exakt reproduzierbar sind und wir die Größe frei wählen können.

Gemessen wird mit `time.perf_counter()`, der präzisen Stoppuhr von Python: Uhr starten, Funktion ausführen, Uhr stoppen. Weil auf einem Rechner immer auch anderes läuft (Betriebssystem, Browser, Hintergrunddienste), messen wir mehrfach und nehmen den **Bestwert**: die schnellste beobachtete Ausführung unter den gegebenen Bedingungen. Er zeigt, was die Funktion kann, wenn wenig dazwischenfunkt; eine typische Laufzeit im Alltag liegt eher etwas darüber. Genau das tut die Funktion `measure`.

Noch ein Hinweis zu den Zahlen, die im Skript unter einigen Zellen abgedruckt sind: Sie sind **exemplarisch**, gemessen auf einem Referenzrechner. Führen Sie die Zellen selbst aus; Ihre Werte werden abweichen, auf die Größenordnungen kommt es an.

```
import time
① rng = np.random.default_rng(42)                                # fester Seed -> reproduzierbar
X = rng.standard_normal((10_000, 384)).astype(np.float32)        # der Messsatz: [10000, 384]
② def measure(f, *args, repeats=3):
    # f mehrfach ausführen und die jeweils gestoppte Zeit sammeln
    ③ times = []
    for _ in range(repeats):
        start = time.perf_counter()
        f(*args)
        times.append(time.perf_counter() - start)
    ④ return min(times)                                           # Bestwert, siehe Text oben
```

① Zahlen dürfen zur Lesbarkeit mit Unterstrich gegliedert werden: `10_000` ist einfach 10000

② Funktionen sind in Python normale Werte: `measure` bekommt die zu messende Funktion `f` als Argument übergeben; `*args` sammelt alle weiteren Argumente ein, egal wie viele

③ `[]` legt eine leere Liste an; `.append` hängt bei jeder Wiederholung die gestoppte Zeit hinten an

④ `min` liefert den kleinsten Wert der Liste — unseren Bestwert

**Ebene 1** ist der naive Weg: Python-Schleifen, wie man sie im ersten Semester schreibt. Jede einzelne der 3,84 Millionen Zahlen wird als Python-Objekt angefasst, Zeile für Zeile die Länge berechnet, dann jede Zahl durch diese Länge geteilt. Das funktioniert, aber Sie werden die Wartezeit spüren (deshalb messen wir hier nur einmal):

```
# Ebene 1 – reines Python: jede Zahl einzeln anfassen
def normalize_python(X):
    result = []
    for row in X:
        ① length = sum(x * x for x in row) ** 0.5      # Länge der Zeile
        ② result.append([x / length for x in row])     # jede Zahl durch die Länge teilen
    return result

t_python = measure(normalize_python, X, repeats=1)
print(f"{t_python:.3f} s")
```

① `sum(x * x for x in row)` summiert die Quadrate, ohne eine Zwischenliste anzulegen (ein Generator-Ausdruck); `** 0.5` heißt „hoch 0,5“, also Quadratwurzel

② `[x / length for x in row]` baut die neue Liste in einer Zeile auf (eine List Comprehension): jedes Element, geteilt durch die Länge  
0.814 s

Eine Fußnote für genaue Leser: Ganz „rein“ ist dieses Python nicht, und effizient auch nicht. Die Schleife läuft über die Zeilen des NumPy-Arrays, und jede entnommene Zahl ist dabei ein NumPy-Skalar, dessen Operationen etwas anderes kosten als die eines echten Python-`float` (wer es sauber will, wandelt vorher mit `X.tolist()` in Python-Listen um). Für unseren Zweck ist das egal: Es geht um den Kontrast zwischen „jede Zahl einzeln anfassen“ und dem, was jetzt kommt — nicht um die bestmögliche Schleife.

### Erst schätzen, dann messen

Ebene 1 brauchte knapp eine Sekunde. Bevor Sie weiterlesen: Um welchen Faktor, schätzen Sie, ist dieselbe Rechnung in NumPy schneller — 2, 20, 200 oder noch mehr? Legen Sie sich auf eine Zahl fest. Dann führen Sie die nächste Zelle aus und vergleichen.

**Ebene 2** rechnet exakt dasselbe, aber als eine einzige NumPy-Zeile. Der Trick: Die Schleifen laufen jetzt nicht mehr im Python-Interpreter, sondern in vorkompiliertem Maschinencode über den ganzen Datenblock auf einmal. Warum das so viel ausmacht, ist das Thema der NumPy-Lerneinheit:

```
# Ebene 2 – NumPy: dieselbe Rechnung, vektorisiert
def normalize_numpy(X):
    return X / np.linalg.norm(X, axis=1, keepdims=True)

t_numpy = measure(normalize_numpy, X)
print(f"{t_numpy * 1000:.2f} ms – Faktor {t_python / t_numpy:,.0f} gegenüber Ebene 1")
```

5.83 ms – Faktor 140 gegenüber Ebene 1

**Ebene 3** ist dieselbe Rechnung in PyTorch, dem Framework, in dem heute die meisten neuronalen Netze trainiert werden. Noch läuft sie auf der CPU, aber schon mit dem Werkzeug, das später nahtlos auf die GPU wechselt:

```
# Ebene 3 – PyTorch auf der CPU (in Colab vorinstalliert)
① try:
    import torch
    Xt = torch.from_numpy(X)
    ② t_torch = measure(lambda: Xt / Xt.norm(dim=1, keepdim=True))
    print(f"{t_torch * 1000:.2f} ms")
except ImportError:
    t_torch = None
    print("PyTorch ist in dieser Umgebung nicht installiert – diese Ebene holen wir in Colab nach.")
```

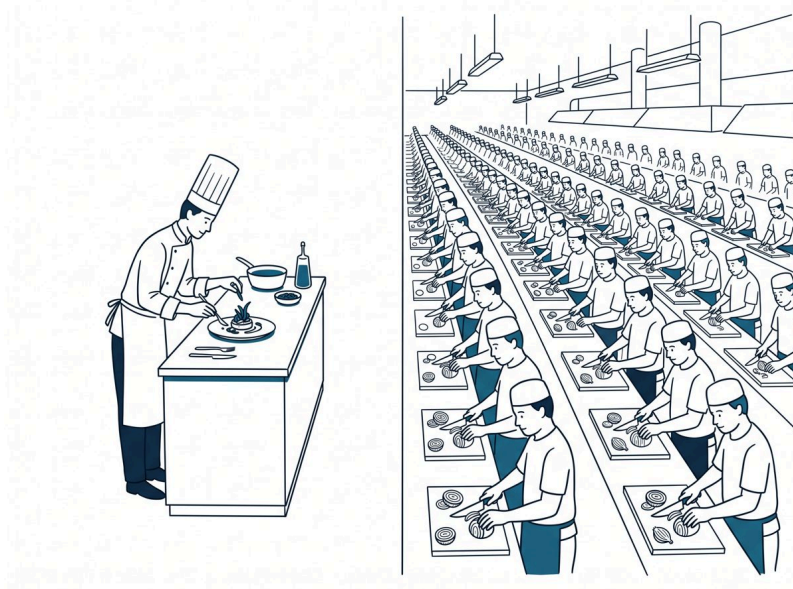
① `try/except` probiert den Block aus und fängt den Fehler ab, statt abzubrechen — hier den `ImportError`, falls PyTorch nicht installiert ist

② `lambda` definiert eine namenlose Mini-Funktion direkt an Ort und Stelle — praktisch, wenn eine Funktion nur einmal gebraucht wird

### Exkurs: CPU und GPU — und was ist CUDA?

Bevor die nächste Ebene die GPU betritt, das Bild dazu. Eine **CPU** hat eine Handvoll sehr schneller, sehr flexibler Kerne — gebaut für abwechslungsreiche Arbeit: Betriebssystem, Browser, unseren Python-Interpreter. Eine **GPU** hat tausende einfacher Kerne, die alle gleichzeitig **dieselbe Rechnung auf verschiedenen Daten** ausführen. Gebaut wurde sie für Grafik — Millionen Pixel, jedes nach derselben Formel —, aber unser Normalize hat genau dieselbe Form: 10.000 Zeilen, jede wird durch ihre Länge geteilt. Wenn Sie so wollen: Die CPU ist ein Spitzenkoch, die GPU eine Halle mit tausend Küchenhelfern, die im Takt Zwiebeln schneiden. Für ein Menü nehmen Sie den Koch; für zehntausend gleiche Handgriffe gewinnt die Halle.

**CUDA** ist der Weg, auf dem Programme diese Halle ansteuern: die Plattform von NVIDIA — Treiber, Compiler und Bibliotheken samt einer C++-Spracherweiterung —, mit der sich GPUs für allgemeines Rechnen programmieren lassen, nicht nur für Grafik. PyTorch spricht unter der Haube CUDA; deshalb ist überall von „CUDA-GPUs“ die Rede, und deshalb schreiben Sie am Ende dieses Semesters selbst einen CUDA-Kernel. Was das praktisch für Ihre Umgebung heißt, steht im Begleitdokument Entwicklungsumgebung (LE01\_entwicklungsumgebung.ipynb); wie GPU und CUDA technisch funktionieren, klärt die GPU-Lerneinheit (M5).



**Ebene 4** ist derselbe PyTorch-Aufruf, nur auf der GPU. Diese Zelle misst live, wenn in Ihrer Umgebung eine CUDA-GPU steckt. Das gilt für eine Colab-GPU-Laufzeit genauso wie für Ihren eigenen Rechner mit NVIDIA-Grafikkarte. Ohne GPU überspringt sie die Messung; die Tabelle unten nutzt dann den vorab gemessenen Wert oder verweist auf die Lerneinheit, in der diese Ebene drankommt. Eine Messentscheidung gehört dazugesagt: Gemessen wird mit Daten, die bereits im GPU-Speicher liegen — das Kopieren dorthin (`Xt.cuda()`) zählt nicht mit:

```
# Ebene 4 – PyTorch auf der GPU (misst nur, wenn eine CUDA-GPU verfügbar ist)
t_gpu = None
if t_torch is not None and torch.cuda.is_available():
    Xg = Xt.cuda() # Messsatz in den GPU-Speicher kopieren
    def normalize_gpu():
        Xg / Xg.norm(dim=1, keepdim=True)
        torch.cuda.synchronize() # auf die GPU warten – warum, zeigt das nächste Kapitel
    normalize_gpu() # einmal aufwärmen (auch das erklärt das nächste Kapitel)
    t_gpu = measure(normalize_gpu, repeats=10)
    print(f"{t_gpu * 1000:.2f} ms auf {torch.cuda.get_device_name(0)}")
else:
    print("Keine CUDA-GPU in dieser Umgebung – die Tabelle nutzt den vorab gemessenen Wert.")
```

Die **Ebenen 5 bis 7** — der PyTorch-Compiler, ein handgeschriebener CUDA-Kernel und die GPU-Sprache Mojo — brauchen Werkzeuge, die Sie erst später im Semester haben. Die nächste Zelle setzt alles zu einer Tabelle zusammen: Wo schon eine Zeit vorliegt (live gemessen oder vorab in Colab), steht sie mit ihrem Faktor gegenüber der Python-Schleife von

Ebene 1. Wo noch keine steht, kommt die Ebene später im Semester dran — dann bauen und messen Sie sie selbst. Jeder Faktor ist gegen die Python-Schleife derselben Messumgebung gerechnet (die Fußzeile sagt, welche Werte vorab gemessen wurden); weil Laufzeiten von Sitzung zu Sitzung schwanken, lesen Sie die Faktoren als Größenordnung:

```
# Live-Messungen und gespeicherte GPU-Messungen zu einer Tabelle zusammensetzen
timings = json.load(open("data/appetizer_timings.json"))

def print_row(name, sec, basis, le=None):
    if sec is not None:
        print(f"{name:28s}{sec * 1000:>9.2f} ms{basis / sec:>9,.0f}x")
    else:
        print(f"{name:28s}{ 'kommt später im Semester':>24s}")

print(f"{'Ebene':28s}{ 'Zeit':>12s}{ 'Faktor':>10s}")
print_row("1 Python-Schleife", t_python, t_python)
print_row("2 NumPy", t_numpy, t_python)
live = {"3": t_torch, "4": t_gpu} # Live-Messungen ersetzen gespeicherte Werte
for name, entry in timings["levels"].items():
    sec = live.get(name[0]) or entry["sec"]
    # Faktor je Zeile gegen die Python-Basis derselben Umgebung rechnen
    basis = t_python if live.get(name[0]) else (timings.get("python_sec") or t_python)
    print_row(name, sec, basis, entry.get("le"))

vorab = [n[0] for n, e in timings["levels"].items() if live.get(n[0]) is None and e["sec"] is not None]
print()
if vorab:
    print(f"Ebene {'', '.join(vorab)}: vorab in Colab ({timings['device']}) gemessen")
if t_gpu is not None:
    print("GPU-Werte live gemessen auf:", torch.cuda.get_device_name(0))
```

| Ebene                 | Zeit                     | Faktor |
|-----------------------|--------------------------|--------|
| 1 Python-Schleife     | 813.95 ms                | 1x     |
| 2 NumPy               | 5.83 ms                  | 140x   |
| 3 PyTorch (CPU)       | 2.99 ms                  | 616x   |
| 4 PyTorch (GPU)       | 0.27 ms                  | 6,886x |
| 5 torch.compile (GPU) | kommt später im Semester |        |
| 6 CUDA-Kernel         | kommt später im Semester |        |
| 7 Mojo                | kommt später im Semester |        |

Ebene 3, 4: vorab in Colab (Tesla T4) gemessen

## Die Reise dieses Semesters

Jede Ebene dieser Tabelle ist eine Station des Semesters. Heute stehen die ersten Zeilen; mit den Lerneinheiten kommen die übrigen dazu, jeweils von Ihnen selbst gebaut und gemessen. Als Vorgeschmack auf die Größenordnung: Zwischen der Python-Schleife und einem guten GPU-Kernel liegt bei dieser Aufgabe grob der Faktor 1.000 bis 10.000, bei identischer Mathematik. In der letzten Lerneinheit kommen wir auf diese Tabelle zurück, schauen uns die Werte noch einmal an und können dann erklären, warum sie so ausfallen.

### 1.1.4 Messen ist schwerer, als es aussieht

Laufzeiten sind die zentrale Währung dieses Moduls, und zugleich erstaunlich leicht falsch zu messen. Zwei klassische Fallen sollen Sie schon heute gesehen haben.

**Falle 1: Eine einzelne Messung ist keine verlässliche Zahl.** Auf jedem Rechner läuft nebenher anderes (Betriebssystem, Browser, Hintergrunddienste), und beim allerersten Aufruf kommen Einmal-Kosten dazu, etwa das Füllen der Caches. Eine Einzelmessung kann deshalb deutlich zu hoch ausfallen. Vergleichen Sie selbst, was eine Einzelmessung gegenüber dem Bestwert aus 20 Wiederholungen meldet (den Erstaufruf-Effekt zeigt diese Zelle nicht mehr, denn `normalize_numpy` ist oben schon gelaufen — den sähen Sie nur in einer frischen Sitzung):

```
# Falle 1: Einzelmessung gegen Bestwert aus 20 Wiederholungen
single = measure(normalize_numpy, X, repeats=1)
best_of = measure(normalize_numpy, X, repeats=20)
print(f"einzelne Messung:   {single * 1000:.2f} ms")
print(f"Bestwert aus 20:   {best_of * 1000:.2f} ms")
```

```
einzelne Messung:   5.14 ms
Bestwert aus 20:   4.23 ms
```

**Falle 2: GPUs arbeiten asynchron.** Ein GPU-Aufruf kehrt sofort zurück: Gerechnet wird im Hintergrund weiter, während Python schon die nächste Zeile ausführt. Wer nur den Aufruf stoppt, misst also das Abschieken der Arbeit, nicht die Arbeit selbst. Bei unserem kleinen Normalize fällt das kaum auf, weil die Rechnung selbst nur Mikrosekunden dauert. Deshalb nehmen wir für die Gegenüberstellung eine deutlich größere Rechnung, eine Matrixmultiplikation. Mit einer CUDA-GPU messen Sie live, ohne kommt das Ergebnis aus der Messdatei:

```
# Falle 2: naive GPU-Messung (ohne Warten) gegen korrekte Messung (mit Synchronisation)
# Eine große Matrixmultiplikation, damit der Unterschied deutlich sichtbar wird
if t_gpu is not None:
    _ = Xg @ Xg.T                                # einmal aufwärmen, damit beide Messungen
    torch.cuda.synchronize()                      # aus demselben Zustand starten (siehe Falle 1)
    start = time.perf_counter()
    Xg @ Xg.T                                     # ohne Warten: stoppt nur das Abschieken
    naive_ms = (time.perf_counter() - start) * 1000
    torch.cuda.synchronize()
    start = time.perf_counter()
    Xg @ Xg.T
    torch.cuda.synchronize()                     # mit Warten: die echte Rechenzeit
    correct_ms = (time.perf_counter() - start) * 1000
    print(f"GPU, naiv gemessen:   {naive_ms:.2f} ms   (ohne Synchronisation)")
    print(f"GPU, korrekt gemessen: {correct_ms:.2f} ms")
else:
    trap = timings["gpu_timing_trap"]
    if trap["naive_ms"] is not None:
        print(f"GPU, naiv gemessen:   {trap['naive_ms']:.2f} ms   (ohne Synchronisation)")
        print(f"GPU, korrekt gemessen: {trap['correct_ms']:.2f} ms")
    else:
        print("Die GPU-Gegenüberstellung wird beim ersten Lauf auf der Kurs-Hardware gemessen.")
```

Wie man auf der GPU korrekt misst, mit Aufwärmen und Synchronisation, lernen Sie in der Lerneinheit zu Tensoren und Messung. Bis dahin gilt die Grundregel dieses Moduls: keiner Laufzeitzahl glauben, deren Messweg Sie nicht kennen.

### 1.1.5 Die Landkarte: von Python zur GPU

Alles in diesem Modul spielt auf einem Weg mit fünf Stationen. Ganz oben schreiben Sie Python; ganz unten rechnet eine GPU. Dazwischen liegen die Schichten, die Python-Code schrittweise in effizienten Maschinen- und GPU-Code übersetzen. Jede Lerneinheit dieses Semesters gehört zu genau einer dieser Stationen — und auch die sieben Ebenen des Appetizers sind nichts anderes als feinere Schritte auf diesem Weg (rechte Spalte der Tabelle).

Die Landkarte taucht in jeder Lerneinheit wieder auf, mit einer Markierung, wo wir gerade sind. Lesen Sie sie von oben nach unten als Übersetzungsweg: Jede Station nimmt den Code der Station darüber entgegen und bringt ihn der Maschine ein Stück näher, und jede Station hat eigene Werkzeuge, eigene Fehlerbilder und eigene Stellschrauben für Geschwindigkeit. Wenn Ihnen draußen eine Technologie begegnet, ist die erste nützliche Frage darum immer dieselbe: Auf welcher Station arbeitet das? TensorFlow beantwortet sie anders als ein handgeschriebener CUDA-Kernel, und wer die Station kennt, weiß schon ungefähr, mit wem es konkurriert und worauf es aufbaut. Produktnamen auswendig lernen müssen Sie nicht; das Einordnen ist die Fähigkeit, um die es geht.

|   | Station                    | Was dort passiert                           | Beispiele             | Ebenen im Appetizer | Mini-Module            |
|---|----------------------------|---------------------------------------------|-----------------------|---------------------|------------------------|
| → | <b>Python &amp; Arrays</b> | bequemer Code, Daten als Arrays             | Python, NumPy, pandas | 1, 2                | <b>M1</b><br><b>M2</b> |
|   | Framework                  | Tensoren, Autograd, fertige Bausteine       | PyTorch               | 3, 4                | <b>M3</b>              |
|   | Compiler                   | Python-Code wird zu schnellem Maschinencode | torch.compile, Triton | 5                   | <b>M4</b>              |
|   | GPU                        | tausende Rechenkerne, eigener Speicher      | CUDA, Mojo            | 6, 7                | <b>M5</b>              |
|   | Skalierung & Serving       | viele GPUs, viele Nutzer                    | DDP, vLLM             | —                   | <b>M6</b>              |

Der Pfeil zeigt, wo wir stehen: Heute und in den nächsten Wochen arbeiten wir auf der obersten Station. Die rechte Spalte zeigt, welches **Mini-Modul** den Stoff der Station trägt — was es mit diesen sechs farbigen Bausteinen auf sich hat, erklärt das Kapitel [1.2](#).

### Lernfragen 1.1.5 #1

- Die Kosinus-Ähnlichkeit zweier Absätze beträgt 0,97, die zweier anderer 0,05. Was sagt das über die beiden Paare aus?
- Welche Aussagen zur Ebenen-Tabelle stimmen?
  - NumPy ist um Größenordnungen schneller als die Python-Schleife, obwohl beide exakt dieselbe Mathematik ausführen
  - Eine GPU lohnt sich immer, auch für kleine Datenmengen
  - Einen eigenen CUDA-Kernel schreiben Sie in diesem Modul selbst
- Drei Technologien, die nicht in der Landkarten-Tabelle stehen: TensorFlow, Numba (übersetzt einzelne Python-Funktionen in Maschinencode) und Kubernetes (verteilt Anwendungen auf viele Rechner). Zu welcher Station gehört jede?

## 1.2 Organisation und Arbeitsweise

**Mini-Module.** Der Stoff ist quer zu den wöchentlichen Terminen in sechs **Mini-Module** gegliedert: thematische Bausteine, die sich über mehrere Lerneinheiten erstrecken: **M1** Python-Grundlagen, **M2** Data-Science-Werkzeugkasten, **M3** Tensoren und Autograd, **M4** Dispatch und Compiler, **M5** CUDA und Performance, **M6** Abstraktion und Skalierung. Jedes Kapitel in Skript und Folien trägt den farbigen Chip seines Mini-Moduls — im Inhaltsverzeichnis oben sind Ihnen die ersten schon begegnet. Welches Mini-Modul welche Station trägt, zeigt die rechte Spalte der Landkarte ([1.1.5](#)); die Reihenfolge der Module folgt ihr von oben nach unten.

**Prüfung.** Das Modul schließt mit einer E-Klausur (90 Minuten, am Rechner) mit praktischem Anteil ab. Notebooks werden nicht abgegeben und nicht bewertet: Die Übungen dienen dem Lernen, nicht der Bewertung. Alle organisatorischen Details und Termine stehen im Moodle-Kurs.

### Die Referenzkarten-Regel

Zu jedem Thema entsteht eine kompakte Referenzkarte mit den wichtigsten Fakten und Mustern. Der gesammelte Kartensatz ist in der Klausur zugelassen. Das ist eine bewusste Entscheidung: Sie müssen keine Syntax auswendig lernen. Die Klausur prüft, ob Sie die Konzepte verstanden haben und mit den Karten arbeiten können.

**KI-Werkzeuge.** In den Übungen sind KI-Assistenten erlaubt und ausdrücklich erwünscht, denn Sie werden im Beruf ständig mit ihnen arbeiten. Nutzen Sie sie aber zum Verstehen, nicht zum Abkürzen: In der Klausur sind sie nicht zugelassen, und wer nur Lösungen kopiert hat, merkt das dort zuverlässig.

**Literatur.** In der Hochschulbibliothek steht ein Semesterapparat zu diesem Modul (oder wird in Kürze dort stehen): ergänzende Literatur zum Nachschlagen, kein Pflichtkauf.

- W. W. Hwu, D. B. Kirk, I. El Hajj: *Programming Massively Parallel Processors. A Hands-on Approach*. 4. Aufl., Morgan Kaufmann, 2022 — das Standardwerk zur GPU-Programmierung mit CUDA.
- L. Ramalho: *Fluent Python*. 2. Aufl., O'Reilly, 2022 — Python in der Tiefe: Datenmodell, Referenzsemantik, idiomatischer Stil.
- J. VanderPlas: *Python Data Science Handbook*. 2. Aufl., O'Reilly, 2023 — NumPy, pandas, Matplotlib und scikit-learn kompakt.
- W. McKinney: *Python for Data Analysis*. 3. Aufl., O'Reilly, 2022 — pandas, erklärt von seinem Erfinder.
- A. Géron: *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. 3. Aufl., O'Reilly, 2022 — praxisnaher Rundgang durchs maschinelle Lernen.
- S. Raschka, Y. Liu, V. Mirjalili: *Machine Learning with PyTorch and Scikit-Learn*. Packt, 2022 — ML-Grundlagen mit PyTorch.
- E. Stevens, L. Antiga, T. Viehmann: *Deep Learning with PyTorch*. Manning, 2020 — PyTorch vom Tensor bis zum Training.

Was wovon relevant ist, sagt Ihnen das jeweilige Skript: Jedes Kapitel nennt seine Quellen.

**Wie dieses Material funktioniert.** Skript und Workbook sind inhaltsgleich aufgebaut und arbeiten mit wiederkehrenden Elementen: Farbige Kästen heben Definitionen (rot), wichtige Aussagen (blau) und Exkurse (grauer Rahmen) hervor. Hinter „Mehr dazu“ verbergen sich optionale Details, hinter „Code Schritt für Schritt“ die zeilenweise Erklärung einer Codezelle; die Kreisnummern am Code-Rand gehören zu den Erklärungen darunter. Einträge mit  sind freiwillige Vertiefungen. Und Prüfzellen führen Sie einfach aus — sie melden, ob Ihre Lösung stimmt.

**Arbeitsumgebung.** Sie arbeiten im Browser in Jupyter-Notebooks. Für die ersten Wochen läuft das über Google Colab oder ohne Konto lokal auf Ihrem Rechner; sobald der GPU-Server der Hochschule bereitsteht, ziehen wir dorthin um, ohne dass sich an den Notebooks etwas ändert. Wie Sie die Notebooks zum Laufen bringen und wie Sie sie bedienen, steht im Begleitdokument Entwicklungsumgebung (`LE01_entwicklungsumgebung.ipynb`). Arbeiten Sie es einmal zu Semesterbeginn durch; der Selbsttest darin sagt Ihnen, was Ihre Umgebung kann.

Kurzfassung für Eilige: Colab-Link aus Moodle öffnen, sofort „**Kopie in Drive speichern**“, Zellen mit Umschalt+Eingabe ausführen. Eine GPU brauchen Sie für die ersten Lerneinheiten noch nicht.

## 1.3 Python und sein Ökosystem

M1 · Python-Grundlagen

Bevor Sie im Lab die ersten Funktionen schreiben, ein kurzer Blick auf die Sprache selbst: woher sie kommt, wie sie ausgeführt wird und warum ausgerechnet sie die KI-Welt dominiert.

**Woher Python kommt.** Entworfen hat die Sprache Guido van Rossum, die erste Version erschien 1991. Der Name stammt von der Komikertruppe Monty Python, nicht von der Schlange. Das Entwurfsziel war von Anfang an gut lesbarer Code. Heute pflegt die Python Software Foundation die Sprache, eine neue Version erscheint jährlich.

**Python 2 und Python 3.** Lange gab es zwei Welten: 2008 erschien Python 3 und räumte an Stellen auf, die mit Python 2 unvereinbar waren (unter anderem Unicode-Strings und `print` als Funktion). Der Umstieg zog sich über ein Jahrzehnt; seit 2020 wird Python 2 nicht mehr gepflegt. Alles Aktuelle, auch dieser Kurs, ist Python 3. Alte Python-2-Beispiele im Netz erkennen Sie meist am `print` ohne Klammern.

**Philosophie.** Die Leitsätze der Sprache stehen im „Zen of Python“ (geben Sie einmal `import this` in eine Codezelle ein): Lesbarkeit zählt, explizit ist besser als implizit, und es sollte einen offensichtlichen Weg geben, etwas zu tun. Daher kommt die Einrückung als Syntax, und daher kommt der kompakte Stil, den Sie im Lab sehen werden. Code, der diese Ideen nutzt, heißt „pythonisch“.

**Wie Python ausgeführt wird.** Python ist eine interpretierte Sprache: Ein Programm, der **Interpreter**, liest Ihren Code und führt ihn Anweisung für Anweisung aus. Einen separaten Übersetzungsschritt, den Sie wie bei C++ selbst anstoßen, gibt es nicht; intern übersetzt der Interpreter den Quelltext automatisch in einen Zwischencode (Bytecode) und führt diesen aus. Die Standard-Implementierung dieses Interpreters heißt **CPython**, sie ist selbst in C geschrieben. Dieser Komfort kostet Geschwindigkeit, und genau das haben Sie in Ebene 1 des Appetizers gemessen. Einige dieser Abläufe unter der Haube schauen wir uns im Laufe des Semesters genauer an.

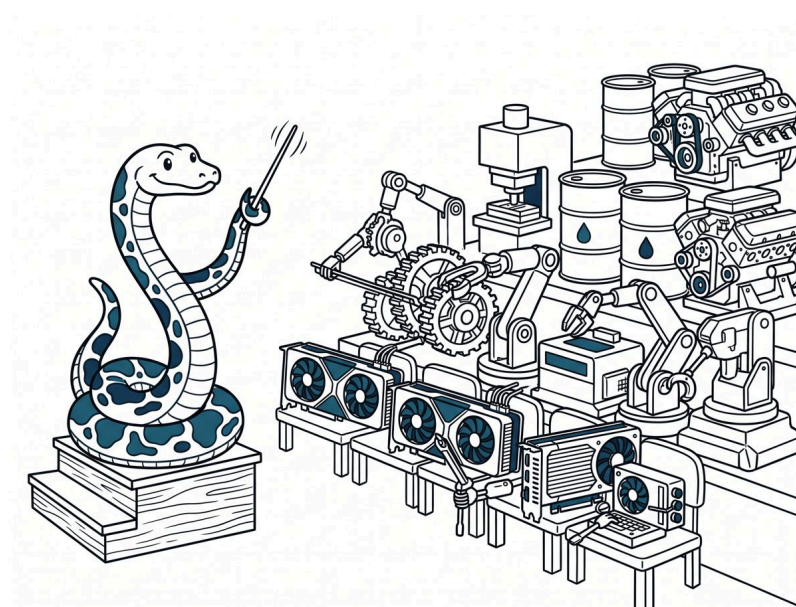
**Pakete und Ökosystem.** Die eigentliche Stärke der Sprache liegt neben ihr: Der Python Package Index (PyPI) hält Hunderttausende fertige Pakete bereit. Installiert werden sie mit einem **Package Manager** (`pip`, oder moderner `uv`, das auch unser Startskript nutzt), üblicherweise in eine **virtuelle Umgebung** je Projekt, damit sich die Abhängigkeiten verschiedener Projekte nicht in die Quere kommen. In Colab und auf dem Hochschul-Server sind die Kurspakete bereits installiert. Für Daten und KI ist dieses Ökosystem konkurrenzlos: NumPy, pandas, Matplotlib, scikit-learn und PyTorch sind allesamt Python-Pakete, und sie decken alle Stationen unserer Landkarte ab.

**Das Zwei-Sprachen-Muster.** Wie passt beides zusammen, ein vergleichsweise langsamer Interpreter und ein Ökosystem, das die rechenintensivste Disziplin der Informatik dominiert? Die Antwort ist ein Muster, das fast alle großen Pakete nutzen: Die Rechenkerne sind in einer kompilierten Sprache geschrieben, und Python dient als komfortable Bediensicht darüber. NumPy rechnet in C, PyTorch in C++ und auf der GPU in CUDA, SciPy greift für lineare Algebra auf jahrzehntelang optimierte Fortran-Bibliotheken zurück, und die Tokenizer der Hugging-Face-Bibliotheken sind in Rust geschrieben. Ein Aufruf wie `np.dot(a, b)` verlässt deshalb den Interpreter und läuft als kompilierter Maschinencode; nur das Ergebnis kommt als Python-Objekt zurück. Sie schreiben also Python, gerechnet wird woanders. In der Fachliteratur heißt diese Arbeitsteilung das **Two-Language Problem**: Wer einen schnellen Rechenkern schreiben will, muss beide Welten beherrschen. Neuere Sprachen wie Julia oder Mojo setzen genau hier an und versprechen, Komfort und Geschwindigkeit in einer Sprache zu vereinen. Für dieses Muster spricht noch ein zweiter Grund: Innerhalb eines Prozesses führt CPython wegen des **Global Interpreter Lock** (GIL) immer nur einen Python-Thread zur Zeit aus (seit Python 3.14 gibt es daneben eine offiziell unterstützte Variante ohne GIL, die sich aber erst verbreiten muss). Parallel rechnen lässt sich trotzdem — mit mehreren Prozessen, oder eben in den kompilierten Rechenkernen, die den GIL während der Arbeit freigeben.

**Wo Python überall steckt.** Datenanalyse und KI sind das dominierende Einsatzgebiet, dazu kommen Web-Backends (Django, FastAPI), Automatisierung und Skripte, wissenschaftliches Rechnen und die Lehre. In Umfragen unter Entwicklerinnen und Entwicklern steht Python seit Jahren auf den vordersten Plätzen.

### Python ist die Steuerzentrale, nicht das Rechenwerk

Python orchestriert schnelle Bausteine, die in anderen Sprachen geschrieben sind. Dass sich beides so sauber verbinden lässt, ist der eigentliche Grund für Pythons Dominanz in der KI, und zugleich der rote Faden dieses Moduls: Wir steigen von der Steuerzentrale hinab zu den Rechenwerken.



**Vertiefung:** Offizielle Anlaufstellen: [python.org](https://python.org) ist die Heimat der Sprache, das [offizielle Python-Tutorial](#) (auch auf Deutsch) führt systematisch durch sie hindurch und eignet sich gut zum Nachschlagen neben dem Grundkurs. Der Zen of Python steht als [PEP 20](#) im Original.

## 1.4 Lab: Funktionen für die Suche

M1 · Python-Grundlagen

Sie bauen die Ausgabeseite der semantischen Suche, mit grundlegenden Python-Werkzeugen und auf echten Daten. Der Appetizer hat die Treffer der Suche fertig formatiert präsentiert; jetzt schreiben Sie genau diesen Teil selbst. Die Suchergebnisse kommen als Liste von Paaren aus Titel und Ähnlichkeitswert. Ihre Funktionen formatieren, filtern und sortieren sie. Nach jeder Aufgabe sagt Ihnen eine Prü fzelle sofort, ob Ihre Lösung stimmt. Wenn eine Prüfung fehlschlägt: Lesen Sie die Meldung, korrigieren Sie Ihre Funktion und führen Sie beide Zellen erneut aus. Und wenn Ihnen die Python-Bausteine dafür noch fehlen, arbeiten Sie zuerst die Python-Grundlagen (LE01\_python\_grundlagen\_1.ipynb) durch, kleinschrittig und mit denselben Mustern wie hier.

### 1.4.1 Die Daten

Das Lab ist eigenständig: Die folgende Vorgabe-Zelle lädt die Daten des Appetizers (falls in dieser Sitzung noch nicht geschehen) und stellt Ihnen die Funktion `get_hits(query_no)` bereit, die für eine Beispielanfrage (0 bis 7) die Trefferliste liefert: eine Liste von Paaren `(title, score)`, unsortiert. Wie die Ähnlichkeitswerte berechnet werden, kommt früh im Semester dran. Heute sind sie einfach da.

```
# Vorgabe: Daten und Trefferlisten bereitstellen – einfach ausführen
import json
import numpy as np

corpus = json.load(open("data/appetizer_passages.json"))
passages = corpus["passages"]
E = np.load("data/appetizer_embeddings.npy")
queries = json.load(open("data/appetizer_queries.json"))["queries"]
Q = np.load("data/appetizer_query_embeddings.npy")

def get_hits(query_no):
    # liefert [(title, score), ...] für die Beispielanfrage query_no – unsortiert
    q = Q[query_no] / np.linalg.norm(Q[query_no])
    scores = (E / np.linalg.norm(E, axis=1, keepdims=True)) @ q
    return [(passages[i]["title"], float(scores[i])) for i in range(len(passages))]

hits = get_hits(0)
print("Anfrage:", queries[0]["text"])
print(len(hits), "Treffer, die ersten drei (unsortiert):", hits[:3])
```

```
Anfrage: Wo liegt San Diego?
400 Treffer, die ersten drei (unsortiert): [('Aufzugsanlage', 0.010074928402900696), ('Sichuan',
0.06887317448854446), ('Antenne', 0.012029776349663734)]
```

Zwei Python-Muster, die Sie in diesem Lab laufend brauchen: Die `for`-Schleife läuft direkt über die Elemente der Liste, ganz ohne Index. Und ein Paar wie `("San Diego", 0.831)` lässt sich dabei gleich in zwei Variablen auspacken. Modernes C++ kennt beides übrigens auch (`for (const auto& [title, score] : hits)`), in Python ist es nur der Normalfall statt der Ausnahme:

```
for title, score in hits:
    ...
```

Mehr zu Paaren (Tupeln) und anderen Containern kommt in der nächsten Lerneinheit, für heute reicht dieses Muster.

## 1.4.2 Treffer formatieren

### Aufgabe 1.4.2 #1

10 min

Schreiben Sie `format_hit(title, score)`, die einen Treffer als eine Textzeile zurückgibt: der Ähnlichkeitswert mit **drei Nachkommastellen**, zwei Leerzeichen, dann der Titel, zum Beispiel `0.831 San Diego`. Nutzen Sie einen f-String mit Formatangabe `{score:.3f}`. Überlegen Sie vor der Prüfwelle: Was gibt Ihre Funktion für den Score `0.1236` zurück?

```
def format_hit(title, score):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle – einfach ausführen
assert format_hit("San Diego", 0.831) == "0.831 San Diego"
assert format_hit("Gehirn", 0.9) == "0.900 Gehirn", "drei Nachkommastellen, auch bei 0.9"
assert format_hit("X", 0.1234) == "0.123 X", "auf drei Stellen runden"
assert format_hit("X", 0.1236) == "0.124 X", "runden, nicht abschneiden"
print("Alle Prüfungen bestanden ✓")
```

## 1.4.3 Treffer filtern

### Aufgabe 1.4.3 #1

10 min

Schreiben Sie `filter_hits(hits, threshold)`, die eine **neue Liste** nur mit den Paaren zurückgibt, deren Score größer oder gleich `threshold` ist. Die Reihenfolge bleibt erhalten, die Eingabeliste wird nicht verändert. Kurz nachgedacht, bevor Sie die Prüfwelle ausführen: Welche Paare der Testliste bleiben bei `threshold=0.5` übrig — und warum zählt der Grenzwert selbst mit?

```
def filter_hits(hits, threshold):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle – einfach ausführen
_test = [("B", 0.5), ("A", 0.9), ("C", 0.1)] # absichtlich unsortiert
_kopie = list(_test)
assert filter_hits(_test, 0.5) == [("B", 0.5), ("A", 0.9)], "0.5 zählt mit (größer ODER gleich), Reihenfolge bleibt"
assert filter_hits(_test, 0.95) == []
assert _test == _kopie, "die Eingabeliste darf nicht verändert werden (auch nicht sortiert)"
assert len(filter_hits(get_hits(), 0.5)) >= 1, "die echte Anfrage 0 hat mindestens einen starken Treffer"
print("Alle Prüfungen bestanden ✓")
```

## 1.4.4 Die besten Treffer finden

### Aufgabe 1.4.4 #1

15 min

Zwei Funktionen, aufeinander aufbauend. Erst `best_hit(hits)`: das Paar mit dem **höchsten** Score, gefunden mit einer Schleife (ohne `sorted` oder `max`). Starten Sie die Suche mit dem ersten Treffer der Liste, nicht mit null — Scores dürfen negativ sein. Dann `top_hits(hits, k)`: gibt **höchstens k** vorhandene Treffer zurück, absteigend sortiert, und lässt die Eingabeliste unverändert. Ein möglicher Weg: höchstens `min(k, len(hits))`-mal `best_hit` auf einer Arbeitskopie aufrufen und den Fund jeweils mit `remove` entfernen.

Dazu drei Python-Bausteine: `rest = hits.copy()` erzeugt eine echte neue Liste; `rest = hits` dagegen klebt nur einen zweiten Namen an dieselbe Liste (siehe Grundkurs, Kapitel „Variablen sind Etiketten“). Und `rest.remove(x)` entfernt das Element `x` aus der Liste. Überlegen Sie kurz: Was würde ohne die Kopie mit der Eingabeliste des Aufrufers passieren?

```
def best_hit(hits):
    # TODO: Ihre Lösung hier
    ...

def top_hits(hits, k):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle – einfach ausführen
_test = [("A", 0.2), ("B", 0.9), ("C", 0.5)]
_kopie = list(_test)
assert best_hit(_test) == ("B", 0.9)
assert best_hit([("A", -0.8), ("B", -0.2)]) == ("B", -0.2), "auch bei lauter negativen Scores (nicht mit 0
starten)"
assert top_hits(_test, 2) == [("B", 0.9), ("C", 0.5)], "absteigend sortiert"
assert top_hits(_test, 10) == [("B", 0.9), ("C", 0.5), ("A", 0.2)], "k darf größer als die Liste sein"
assert _test == _kopie, "die Eingabeliste darf nicht verändert werden"
_echt = top_hits(get_hits(0), 1)[0]
assert _echt[0] == "San Diego", "Anfrage 0 sollte San Diego auf Platz 1 haben"
print("Alle Prüfungen bestanden ✓")
```

### 1.4.5 Alles zusammensetzen

#### Aufgabe 1.4.5 #1

10 min

Jetzt fügt sich alles zusammen: `print_results(query_no, k=3)` gibt die Anfrage und darunter die `k` besten Treffer aus, jede Zeile mit `format_hit` formatiert und um zwei Leerzeichen eingerückt. Nutzen Sie Ihre eigenen Funktionen, neue Logik braucht es hier nicht.

```
def print_results(query_no, k=3):
    # TODO: Ihre Lösung hier
    ...

print_results(0)
```

```
# Prüfwelle – einfach ausführen (prüft die Ausgabe von print_results Zeile für Zeile,
# mit Ihren eigenen, oben bereits geprüften Funktionen als Vergleich)
import io, contextlib

def _pruefe(query_no, k):
    _buf = io.StringIO()
    with contextlib.redirect_stdout(_buf):
        print_results(query_no, k=k)
    _zeilen = [z for z in _buf.getvalue().splitlines() if z.strip()]
    assert _zeilen[0] == "Anfrage: " + queries[query_no]["text"], "erste Zeile: die Anfrage"
    _soll = [" " + format_hit(t, s) for t, s in top_hits(get_hits(query_no), k)]
    assert _zeilen[1:] == _soll, "dann die k besten Treffer, formatiert und mit zwei Leerzeichen
eingerückt"

_pruefe(3, 2)
_pruefe(0, 3)
print("Alle Prüfungen bestanden ✓")
```

Vergleichen Sie Ihre Ausgabe mit der `search`-Funktion im Appetizer: Sie haben gerade denselben Baustein selbst geschrieben. Probieren Sie die übrigen Anfragen (0 bis 7) und verschiedene `k`.

### 1.4.6 Zusatzaufgaben (Fast Track)

Für alle, die schneller fertig sind: ohne Prüfzellen, dafür mit Spielraum.

**Zusatz 1:** Schreiben Sie `mean_score(hits)`, die den Durchschnitt aller Scores berechnet, und vergleichen Sie die Durchschnitte der acht Anfragen: Warum liegen sie alle so niedrig, obwohl die Top-Treffer hoch sind?

**Zusatz 2:** Schreiben Sie `score_bar(score)`, die einen Score als Balken aus `#`-Zeichen darstellt (ein Zeichen je 0.05, also `0.83` → 16 Zeichen), und erweitern Sie `print_results` um diese Balken.

**Zusatz 3:** `second_best(hits)` liefert den zweitbesten Treffer, ohne `top_hits` zu benutzen, nur mit einer einzigen Schleife.

*# Platz für die Zusatzaufgaben*

#### → Nächste Schritte

- Richten Sie Ihre Arbeitsumgebung ein (Begleitdokument Entwicklungsumgebung (`LE01_entwicklungsumgebung.ipynb`)), falls im Termin nicht geschehen. Lassen Sie nach jedem Umgebungswechsel den Selbsttest darin erneut laufen.
- Bearbeiten Sie das Lab (Kapitel [1.4](#)), bis alle Prüfzellen grün sind (Sollzeit rund 2 Stunden). Wenn es hakt: erst die Python-Grundlagen (`LE01_python_grundlagen_1.ipynb`), dann zurück ins Lab.
- Spielen Sie den Appetizer durch: alle acht Beispielanfragen, andere `k`-Werte.

→ **Ausblick:** Weiter geht es in der nächsten Lerneinheit mit Python II: Container, Slicing und Klassen. Dort bekommen die Paare von heute ihren richtigen Namen.

#### Quellen und Referenzen

- Python Software Foundation: [python.org](https://python.org) und die [Python-Dokumentation](#) — offizielle Referenz zu Sprache und Standardbibliothek, Grundlage des Kapitels „Python und sein Ökosystem“.
- J. R. Firth: *A synopsis of linguistic theory 1930–1955*. In: *Studies in Linguistic Analysis*, Oxford, 1957 — Ursprung des „company it keeps“-Prinzips.
- T. Mikolov u. a.: *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781, 2013 — das word2vec-Paper, Quelle der Analogie-Beispiele (König – Mann + Frau).
- N. Reimers, I. Gurevych: *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019 — die Modellfamilie hinter dem Kurs-Embeddingmodell.
- Datengrundlage von Appetizer und Lab: [GermanQuAD](#) (deepset), Lizenz CC BY 4.0; die Absatztexte stammen aus der deutschen Wikipedia (CC BY-SA). Attribution auch in `data/appetizer_passages.json`.
- Embedding-Modell: paraphrase-multilingual-MiniLM-L12-v2 (Sentence-Transformers), Apache 2.0.